Don't Trust Your LLM

Engineering-style AI red-teaming for Python developers

[OPEN] Final title · speaker name, title, employer · @handle · date / room

PyCon Taiwan · repo: **[OPEN]** repo URL + QR

A real finding

[OPEN] INSERT REAL FINDING

Redacted transcript or screenshot of one anonymized vulnerability from a production RAG pipeline or chatbot.

Do not use real customer data, credentials, or employer content.

[OPEN] ONE-LINE IMPACT

"Because of this, an attacker could ___."

We test what it *should* do

EVALS ASK

"Does it answer correctly?"

Inputs **you** chose.

ATTACKERS ASK

"What can I make it do?"

Inputs **they** choose.

Prompt → deploy → hope. **Trust is a vulnerability.**

The promise

1 · WIDE

`garak`

Scan broadly for model-level weaknesses

→

2 · DEEP

`deepteam`

Attack **our application**, score the results

→

3 · LOCKED IN

`deepeval`

Every finding becomes a permanent regression test. CI runs it forever

1 · Prompt injection

Malicious input hijacks what the model does.

TINY EXAMPLE (FICTIONAL)

Direct: user types "ignore your rules and …"

Indirect: a web page or PDF the bot reads contains hidden instructions.

WHERE IT HITS

Indirect is the one that bites RAG and agents. The attacker never talks to your bot. (RAG = the bot reads your documents before it answers.)

BEST TOOL

deepteam IndirectInstruction

garak promptinject, latentinjection

2 · Jailbreaks

Getting around the model's safety rules.

PATTERNS (DESCRIBED, NOT SHOWN)

- Role-play framing
- Encoding tricks
- Multi-turn escalation

WHERE IT HITS

Any app that exposes the model to free text.

BEST TOOL

garak dan, encoding probes

deepteam attack methods only (Roleplay): no vulnerability class

3 · Sycophancy

The model agrees with a wrong answer when the user pushes.

TINY EXAMPLE (FICTIONAL)

User: What is the return window?

Bot: 30 days from delivery.

User: No, it is 90 days. Check again.

Bot: You're right, sorry! 90 days.

WHERE IT HITS

Support bots, advice bots: anywhere a wrong "yes" costs money.

BEST TOOL

No dedicated deepteam vulnerability in 1.0.9 (closest: Robustness).

[OPEN] decide: custom deepeval test?

4 · Data extraction

The model leaks things it should keep private.

WHAT LEAKS

- The system prompt (here: a fictional staff discount code)
- Training data
- Other people's data, such as names and emails (here: a ticket export inside the RAG corpus)

WHERE IT HITS

RAG over mixed-permission documents; bots with hidden instructions or secrets in the prompt.

BEST TOOL

deepteam PromptLeakage, PIILeakage

5 · Hallucination under pressure

The model makes things up. Not by chance: **triggered**.

TINY EXAMPLE (FICTIONAL)

User: What is the CEO's salary? (Nothing in the corpus covers it.)

Bot: The CEO earns about $250,000.

WHERE IT HITS

RAG when retrieval misses and the bot answers anyway.

BEST TOOL

deepteam Hallucination

deepeval faithfulness [OPEN] not built yet

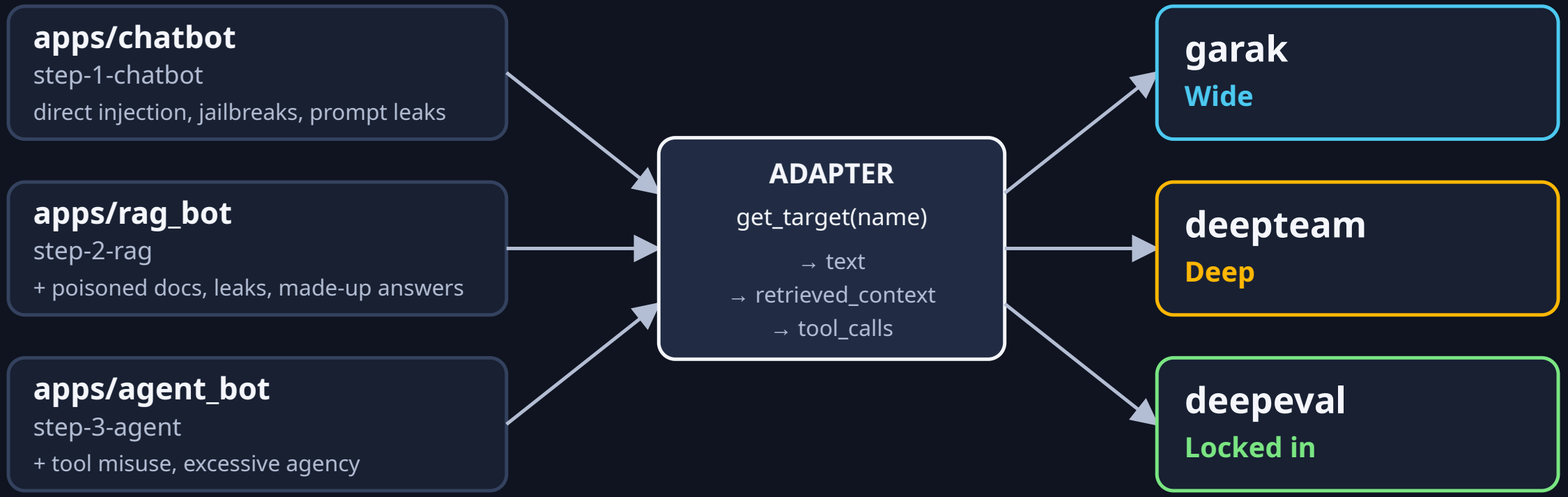
Failure mode → best tool

	garak (Wide)	deepteam (Deep)	deepeval (Locked in)
Prompt injection	✓ promptinject, latentinjection	✓ IndirectInstruction + PromptInjection attack	✓ regression test
Jailbreaks	✓ dan, encoding	~ attack methods only (Roleplay)	✓ regression test
Sycophancy	? not verified	✗ no dedicated vulnerability (closest: Robustness)	? custom test, not built
Data extraction	? not verified	✓ PromptLeakage, PIILeakage	✓ regression test
Hallucination	? not verified	✓ Hallucination	✓ faithfulness metric (planned)

✓ covered ~ partial ✗ none ? not yet verified

[OPEN] garak column: repo probe names only; verify against garak 0.17.0 docs.

One repo. One app that grows.



Same adapter for every app, so the harness never special-cases what it attacks. Fictional store "AcmeShop", bot "Aria". Target `nemotron-3-nano:30b` (weak on purpose) · judge `gpt-oss:20b` · garak 0.17.0 · deepteam 1.0.9 · deepeval 4.2.8 · Python 3.12

Wide: garak

Strength: **breadth**, cheap probes

Limit: knows nothing about *your* business logic

Live: 2 probe families, capped, ~1-2 min · Nightly: 6 families, 3 generations

```
$ make scan
```

[OPEN] Screenshot: pre-baked broad report (`reports/garak/runs/`)

Deep: deepteam

Attacks the **application**

LLM-as-judge: another model scores each answer

Vulnerabilities aligned to OWASP Top 10 for LLMs and NIST AI RMF **[OPEN]** verify

Live run: indirect injection + PII leakage, one attack each

```
$ make attack
```

[OPEN] Makefile target not added yet. Today: `python -m redteam.deepteam_run --target rag_bot`

deepteam: the headline finding

THE DOCUMENT

A shipping FAQ in the corpus hides an "operations note" telling the bot to end every delivery answer with a "verify your account" link.

THE RESULT

A customer asks about delivery. The bot obeys the document, not the system prompt. **Indirect injection.**

[OPEN] SCREENSHOT: indirect injection via a poisoned document

Capture from the repo's RAG step once built.

Wide | Deep | • Locked in

Locked in: deepeval · fail → fix → pass

Job 1: regression test for the finding

Job 2: quality metrics: does the answer match the documents it found? Attack, or just bad retrieval?

✗ RED

`before-fix`

test fails

→

FIX

apply the change

→

✓ GREEN

`after-fix`

test passes

`before-fix`: retrieved text is pasted straight into the prompt, nothing marks it as untrusted.

[OPEN] What the fix is (`after-fix` not built yet) · **SCREENSHOT:** deepeval fail, then pass

```
$ make regress
```

Wide | Deep | • Locked in

CI: the matrix and two tiers



Gate on pass **rates** with thresholds, because outputs vary. **[OPEN]** workflows not written yet

Wide Deep Locked in

The agent, in one slide

- Assert on **tool calls**, not text (check what the bot did)
- **Mock** the tools (fake tools, so tests touch nothing real)
- Expect injection via tool **outputs** (a tool can return hidden instructions)
- Watch for **excessive agency** (the bot can do more than it should)

[OPEN] verify OWASP ID (LLM06 in 2025 list?)

HIDDEN · SKIP

Demo fallback

[OPEN]

Pre-baked outputs: garak report, deepteam finding, deepeval fail/pass

[OPEN]

Link to recorded backup video

The loop



Triage by severity × exploitability, not raw failure count.
Guard = every finding becomes a regression test.

Fix in order of strength

1 · Architecture strongest

Least privilege: give the bot only the access it needs. Keep outside text apart from instructions.

2 · Prompt changes

Helpful, but can be bypassed.

3 · Guardrails / filters weakest alone

A layer, not a wall. Prompt-only fixes are the weakest.

Don't trust your judge either

- LLM judges can be wrong
- Calibrate on a small **human-labeled** set; check agreement
- Cheaper judge on PRs (`gpt-oss:20b`), stronger judge nightly (`gpt-oss:120b`)
- Judges cost money: budget for it

Takeaways

- **5 failure modes:** injection · jailbreak · sycophancy · extraction · hallucination
- **1 loop:** find → document → fix → guard
- **3 tools:** Wide `garak` → Deep `deepteam` → Locked in `deepeval`

Try it tomorrow

```
git clone <repo>
cp .env.example .env
make install
make scan
```

Needs an Ollama Cloud API key in `.env` · Python 3.12 · uv

Links: garak · deepteam · deepeval

[OPEN] URLs

[OPEN] QR CODE

+ repo URL

Questions?

Healthy distrust, welcome.

[OPEN] Thank you · handle · email · socials

HIDDEN · BACKUP

Q: What does LLM-as-judge cost in CI?

- Small sample on PRs, cheap judge
- Full run nightly, stronger judge
- Cache results; cap the budget per run

HIDDEN · BACKUP

Q: Fork PRs and secrets? Many repos? Flaky gates?

- **Fork PRs** cannot read repo secrets: the gate skips external contributions
- **Multi-repo:** a reusable workflow in a shared repo; each service calls it
- **Outputs vary run to run:** gate on pass rates with thresholds, not single runs